LMS Platform

Technical Report: Requirements, Architecture and Assessment

Subject	Learning Management System (monorepo from LMS.zip)
Scope	Backend API, three web frontends, database schema, configuration, risks
Method	Static review of source code and configuration. The application was not executed or tested.
Note	The archive has no formal requirements document. Requirements are inferred from the code.
Date	30 October 2026

1. Executive Summary

The LMS is a multi-role e-learning platform delivered as one repository: a single Flask REST API on PostgreSQL, and three Next.js web applications for administrators, instructors and students. It covers the full learning lifecycle: course authoring, video lessons, enrolment, quizzes and assignments, live sessions, discussions, certificates with public verification, multi-gateway payments including EMI, analytics and audit logging.

Metric	Value
Backend code	about 54,000 lines of Python; 31 route files, 30 model files
API surface	about 277 route handlers under /api/v1
Frontend code	about 75,000 lines of JS/JSX across three Next.js apps
Database	31 PostgreSQL tables (schema.sql)
User roles	Super Admin, Admin (incl. employees), Instructor, Student

Overall assessment: functionally broad and feature-rich. The main concerns are security hygiene (committed secrets, hard-coded fallbacks, long-lived tokens), no automated tests or deployment tooling, and duplicated frontend code.

2. System Architecture

Component	Technology	Port
Backend API	Flask 3.0.2, Flask-SQLAlchemy 3.1.1, Flask-JWT-Extended 4.6.0, Marshmallow, APScheduler 3.10.4, gunicorn 22	5000
Admin portal (admin + super-admin)	Next.js 14.2.35, React 18.3.1, Tailwind, Recharts, ReactFlow, xlsx, jsPDF	3000
Instructor portal	Next.js 14.2.35, React 18.3.1, Recharts, JSZip, xlsx, jsPDF	3001
Student portal	Next.js 14.2.35, React 18.3.1, Recharts, country-state-city	3002
Database	PostgreSQL via psycopg2 (SQLite in-memory for testing config)	5432

- **Single unified API** at */api/v1*, with 30+ blueprints registered in the app factory. All frontends consume it.
- **Proxy layer:** each Next.js app exposes */api/proxy/[...path]* to forward browser requests to the backend; the admin app also proxies uploaded files.
- **Orchestration:** *node start.js* detects Python and the LAN IP, prepares the database, starts the backend and three frontends, and opens the admin portal.
- **Background jobs:** APScheduler runs daily analytics aggregation and a per-minute audit-log job inside the app process.
- **Reporting libraries:** ReportLab, PyMuPDF, Pillow and qrcode for PDFs, certificates and QR codes.

3. Roles, Authentication and Authorisation

- **Roles:** Super Admin, Admin (employee accounts map to admin), Instructor, Student, each with its own signup/signin tables.
- **Tokens:** JWT in the Authorization Bearer header. Access tokens default to 30 days, refresh tokens to 7 days.
- **Enforcement:** a *roles_required* decorator checks the role claim. If missing, it falls back to a database lookup, then to the ID prefix.
- **Onboarding:** employee registration requests go to admins for approval; employees complete onboarding through an emailed OTP. Bulk-import scripts exist.
- **Password flows:** forgot/reset password, OTP for password change, and notification emails to instructors.

4. Functional Requirements by Module

Module	Capabilities
Course management	Courses with level, duration, price, instructor, status, thumbnail, overview video, syllabus, tags and categories. Instructors author; admins manage all. Public catalogue and detail pages.
Lessons and video	File and video lessons, custom video player (progress bar, settings, controls). Uploads up to 5 GB per request, study materials up to 15 GB. Optional ffprobe codec validation.
Enrolment and progress	Enrolment with access status and admin blocking. Per-lesson progress tracking, continue-learning and dashboard views.
Course interest	Students request access; admins approve or reject. Approval can create the account and enrolment. COD guard for cash-style requests.
Quizzes and assignments	Built-in (four options, points, publish/free flags) or external-link quizzes with scored attempts. Assignments with submissions and instructor review.
Live sessions	Scheduled sessions with meeting link and ID, status and runtime status, actual start and end times.
Discussions and announcements	Student course discussions, separate instructor-to-instructor forum, announcements.
Feedback	Students rate instructors; instructors give feedback on students; PDF feedback report.
Certificates	Admin-uploaded PDF templates and configurable certificate types. Eligibility engine; assign, revoke, reinstate. QR code and public verification page. PDF generation with ReportLab, PyMuPDF, Pillow.
Payments	Razorpay, Stripe, PayU, plus mock and demo handlers. Card, UPI, bank transfer with proof upload, COD and EMI. Partial payments, discounts, admin verification. Multi-step student checkout.
EMI	Plan selection and summary; per-installment tracking with retry count, next retry date, failure reason and admin notes.
Notifications and email	In-app notifications and SMTP email (Gmail by default) for account creation, registration, OTPs and password events.
Analytics and reporting	Daily aggregation into lms_analytics (users, courses, enrolments, revenue, payments, ratings, completion). Real-time analytics, Recharts dashboards, Excel/PDF/ZIP export. Student, instructor-performance and employee watch reports.
Audit logging	Middleware and decorator capture admin actions; scheduler job; viewer pages for admin, instructor and super-admin.
Resource library	Admin and team libraries for PDFs, videos, images and office files, organised by department.
Other	Demo bookings, dark/light theme, AI assistant component, mind-map view, phone input with country selection, language context.

5. Data Model

schema.sql defines 31 tables. Identifiers are string-based and generated through an *id_sequences* table. Several modules also self-heal their schema at startup (for example employee registration).

Domain	Tables
Identity	admin_signup, admin_signin, instructor_signup, instructor_signin, student_signup, student_signin, super_admins, users
Learning	courses, lessons, enrollments, progress, quizzes, assignments, discussions, live_sessions, announcements
Commerce	payments, emi_installments, payment_gateway_transactions, course_interest_requests
Certificates	certificates, certificate_templates, certificate_types

Domain	Tables
Platform	notifications, uploaded_documents, audit_logs, lms_analytics, instructor_feedback, instructor_to_student_feedback, demo_bookings, id_sequences

Key payment fields: original, discount, final, paid and remaining amounts; payment type, method and gateway; EMI tenure, monthly amount and next due date; partial-payment percentage; proof URL; and verified-by/at. Certificates carry type, status, QR data, verification URL and revoke/reinstate audit fields.

6. API Surface

Blueprints are mounted under */api/v1/*. Approximate handler counts for the largest areas:

Area	Routes	Area	Routes
Super-admin management	24	Quizzes	13
Admins	21	Courses	12
Admin resources	17	Live sessions	12
Certificates	16	Auth / Users	10 / 10
Payments	15	Uploads / Discussions	9 / 9
Analytics	14	Assignments	8

Other areas: lessons, progress, enrolments, feedback, instructor-discussions, notifications, announcements, statistics, student, instructor, course-interest, demo-bookings, certificate-templates and audit-logs.

7. Frontend Applications

- **Admin (about 60 pages):** courses, students, instructors, enrolments, payments, analytics, quizzes, feedback, course interests, employee requests, resources and team resources; super-admin dashboard, admins, audit logs and analytics; auth pages including employee onboarding.
- **Instructor:** dashboard analytics, course create/edit/lessons, quizzes, assignments, live sessions, discussions, feedback, student report, certificates, submission reviews.
- **Student:** catalogue, my courses, lessons, quizzes, assignments, live sessions, discussions, certificates, resources, feedback, profile, notifications, pricing, demo booking, checkout (card, COD, EMI) and a rich dashboard (achievements, activity feed, calendar, to-do, recommendations, payment summary).

8. Configuration and Environment

Area	Settings
Required	JWT_SECRET_KEY (mandatory outside development; dev falls back to a default and warns)
Database	DATABASE_URL, or USE_LOCAL_DB with LOCAL_DATABASE_URL
Server / CORS	SERVER_HOST, SERVER_PORT, FLASK_ENV, API_BASE_URL, CORS_ORIGINS, DEV_LAN_IP
Uploads	UPLOAD_FOLDER, UPLOAD_EXTRA_ROOTS, MAX_CONTENT_LENGTH_BYTES, MAX_MATERIAL_SIZE_BYTES, FFPROBE_PATH
Certificates	CERTIFICATE_TEMPLATE_PATH, CERTIFICATE_TEMPLATES_UPLOAD_DIR
Email	SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD, SMTP_FROM_EMAIL, SMTP_FROM_NAME, SMTP_USE_TLS
Payments	RAZORPAY_KEY_ID/SECRET, STRIPE_SECRET_KEY, PAYU_MERCHANT_KEY/SALT
Other	JWT_ACCESS_DAYS, JWT_REFRESH_DAYS, PAGINATION_DEFAULT_LIMIT (20), PAGINATION_MAX_LIMIT (100)

9. Non-Functional Characteristics

- **Prerequisites:** Python 3.12+, Node.js, PostgreSQL; ffprobe optional.
- **Performance:** connection pooling with pool_pre_ping and 1-hour recycle; pagination; daily pre-aggregated analytics.
- **Scheduling:** in-process APScheduler. Under several gunicorn workers, jobs could run more than once.
- **Deployment:** gunicorn is listed, but no Dockerfile, CI pipeline or reverse-proxy configuration is included.
- **Testing:** no automated test suite. Only manual smoke/workflow scripts, a DB connection check and an SMTP test.

10. Risks and Findings

#	Severity	Finding	Detail
1	High	Secrets in archive	backend/.env is included and holds database, JWT and SMTP credentials despite being git-ignored. Rotate them and remove the file from any history.
2	High	Hard-coded DB fallback	config.py defaults to a postgres URL with an embedded password.
3	Medium	Token and CORS policy	30-day access tokens; CORS_ORIGINS defaults to *.
4	Medium	Role inference by ID prefix	roles_required falls back to the user ID prefix when the claim and database lookup fail.
5	Medium	Verbose auth logging	Full JWT claims are printed and logged on every protected request.
6	Medium	Payments not reviewed in depth	Webhook signature verification and secret handling need a dedicated review before production.
7	Medium	No tests or CI	No pytest suite, CI pipeline or containers; regressions are likely as the code grows.
8	Low	Duplicate scheduler runs	In-process jobs repeat under multi-worker gunicorn.
9	Low	Duplicated frontend code	Navbar, Sidebar, Card and others are copied across three apps. courses and quizzes keep service.py and service_new.py side by side.
10	Low	Mixed table design	quizzes stores definitions and attempt answers together; models_attempts.py suggests a migration was begun.

11. Recommendations

- Rotate all credentials, remove .env from the archive and history, and drop the hard-coded fallback.
- Shorten access-token lifetime, use refresh tokens, restrict CORS to known origins, and require the role claim.
- Remove verbose claim logging; use structured logging with redaction.
- Add pytest coverage for authentication, payments (including webhooks), EMI and certificate eligibility; add CI.
- Containerise the stack (Postgres, backend, three frontends) and run the scheduler in a single dedicated process.
- Extract shared UI into a common package and retire the duplicated service files.
- Split quiz definitions from attempts and complete the migration.